

Dendê Eventos: Refatoração do Código e Prototipação UI/UX

Autores: Darlan Deivid S. Silva,
Dylan C. Soares,
Leonardo F. C. Nery

Agenda

O objetivo dessa apresentação é
....

1. Nova Arquitetura:

Estrutura final de arquivos
e Model.kt

2. Camada de Dados:

Operações do repositório
e Componentes de
segurança

3. Roteamento:

Refatoração da Main.kt e
os Submenus

4. Logica de Negócio:

Demonstração de 4
funções refatoradas

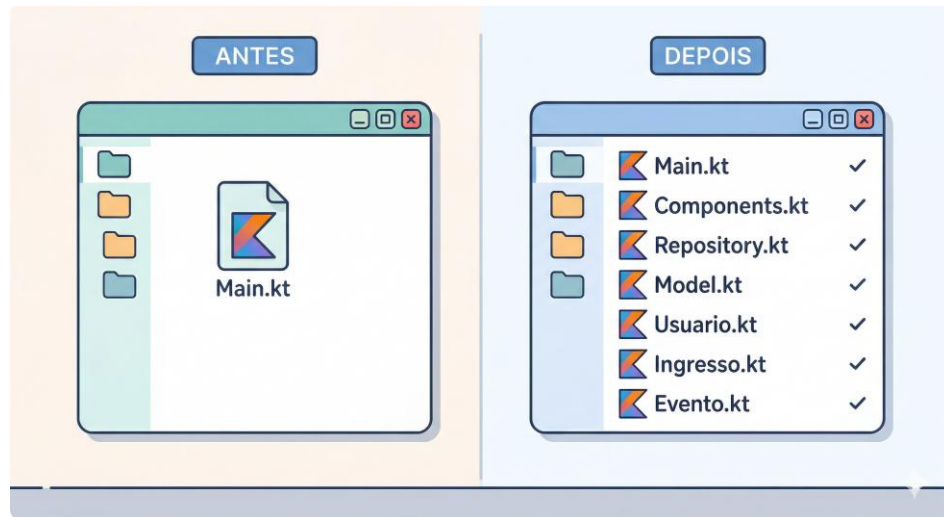
5. Prototipo de Tela UI/UX:

Prototipo das telas Feed
de eventos e Lista de
eventos

Modularização e Modelagem de Dados

- **Fim do Código Monolítico:** O projeto foi dividido por contexto de negócio (**Usuario.kt**, **Evento.kt** e **Ingresso.kt**), transformando cada User Story numa função independente.

- **O Arquivo Model.kt:** Isolamento total das estruturas de dados.



Centralização de Dados e Validações

- **Padrão Repository (Repositorio.kt):**

- Listas mutáveis isoladas.

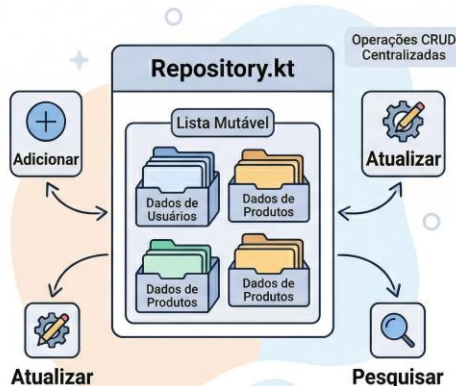
- Funções de **CRUD** (Adicionar, Atualizar, Buscar) centralizadas.

- **Componentes Seguros (Components.kt):**

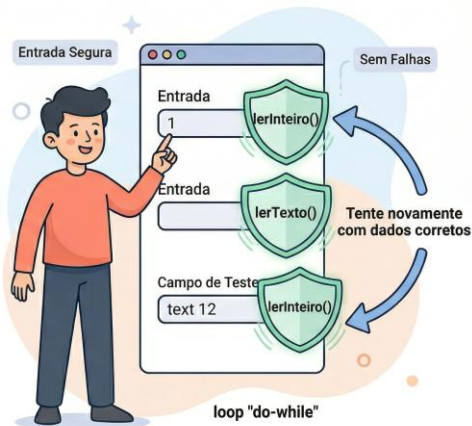
- Substituição de múltiplos try-catch por funções padrão (`readInt`, `readString`).

- Uso de laços do-while para prender o fluxo até o usuário digitar o dado correto, evitando o *crash* do sistema.

Padrão de Repositório (Repository.kt)



Componentes Seguros (Components.kt)



Roteamento e Submenus

- **O Main como Maestro:** O arquivo principal agora não processa mais lógica de negócio, apenas controla o estado do usuário (Logado ou Não).

- **Delegação de Tarefas:** Cada escolha no menu apenas invoca funções externas (Ex: `verPerfil(usuarioLogado)`).

- **Submenus Dinâmicos:** A interface do terminal se adapta ao perfil da sessão (Usuário Comum vs. Organizador), garantindo acesso seguro às *User Stories*.



Exemplo do Main



```
when (opcao) {  
  1 -> cadastrarUsuarioComum()  
  2 -> cadastrarOrganizador()  
  3 -> usuarioLogado = fazerLogin()  
  
  4 -> if (usuarioLogado != null) verPerfil(usuarioLogado) else println("Você precisa estar logado.")  
  5 -> if (usuarioLogado != null) usuarioLogado = alterarPerfil(usuarioLogado) else println("Você precisa estar logado.")  
  6 -> if (usuarioLogado != null) usuarioLogado = inativarConta(usuarioLogado) else println("Você precisa estar logado.")  
  7 -> reativarConta()  
  
  8 -> if (usuarioLogado?.tipo == TipoUsuario.ORGANIZADOR) cadastrarEvento(usuarioLogado) else println("Acesso negado.")  
  9 -> if (usuarioLogado?.tipo == TipoUsuario.COMUM) comprarIngresso(usuarioLogado) else println("Acesso negado.")  
  
  10 -> {  
    usuarioLogado = null  
    println("Logout realizado com sucesso!")  
  }  
  11 -> println("Saíndo do Sistema...")  
  
  12 -> if (usuarioLogado?.tipo == TipoUsuario.ORGANIZADOR) meusEventos(usuarioLogado) else println("Acesso negado.")  
  13 -> if (usuarioLogado?.tipo == TipoUsuario.COMUM) feedEventos(usuarioLogado) else println("Acesso negado.")  
  14 -> if (usuarioLogado?.tipo == TipoUsuario.COMUM) meusIngressos(usuarioLogado) else println("Acesso negado.")  
  
  else -> if (opcao != 11) println("Opção inválida.")  
}
```

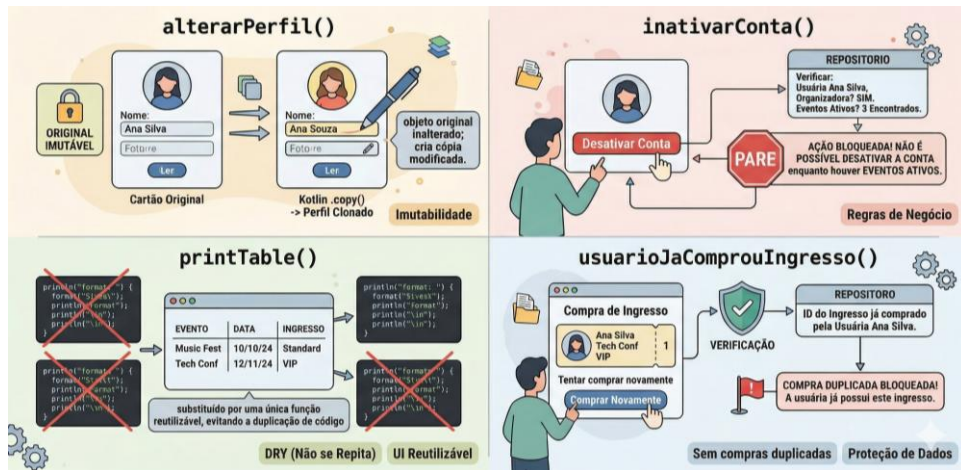
Destakes da Refatoração

- **alterarPerfil():** Refatorada para usar o método `.copy()` do Kotlin, garantindo a atualização do clone sem ferir a imutabilidade do Model.

- **inativarConta():** Implementação da regra de negócio que consulta o repositório para impedir que organizadores com eventos ativos inativem o perfil.

- **printTable():** Componente de interface visual criado para listar eventos e ingressos de forma tabular, reduzindo repetição de código (DRY).

- **usuarioJaComprouIngresso():** Lógica transferida para o Repositório, blindando o sistema contra compras duplicadas para o mesmo evento.



Exemplos das funções



```
val usuarioAtualizado = usuario.copy(  
    nome = novoNome,  
    sexo = novoSexo,  
    senha = novaSenha  
)
```

```
fun usuarioJaComprouIngresso(emailUsuario: String, nomeEvento: String): Boolean{  
    return ingressos.any {  
        it.emailUsuario.equals(emailUsuario, ignoreCase = true) &&  
        it.nomeEvento.equals(nomeEvento, ignoreCase = true) &&  
        !it.cancelado  
    }  
}
```


Exemplos das funções



```
if (confirmacao == "SIM") {  
    //Organizador não pode inativar sua conta se tiver eventos ativos  
    if (usuario.tipo == TipoUsuario.ORGANIZADOR) {  
        if (organizadorTemEventoAtivo(usuario.email)) {  
            println("ERRO: Você possui eventos ativos! Desative seus eventos antes de inativar a conta")  
            return usuario  
        }  
    }  
}  
  
//Altera o status do usuário para inativo e desloga ele  
val inativado = usuario.copy(ativo = false)  
atualizarUsuario(usuario, inativado)  
println("Conta inativada com sucesso! Você foi desconectado.")
```

Exemplos das funções



```
fun printTable(header: String, items: List<Any>) { Leonardo Freitas
    // Cria uma linha separadora com base no tamanho do título
    val tracos = "-".repeat(n: header.length + 20)

    println("\n$tracos")
    println(" $header")
    println(tracos)

    if (items.isEmpty()) {
        println(" Nenhum registo encontrado.")
    } else {
        // Percorre a lista genérica (Any) e imprime cada item. O Kotlin usa o .toString() das Data Classes automaticamente.
        items.forEach { item ->
            println(" $item")
        }
    }
    println("$tracos\n")
}
```

Prototipação UI/UX

- Design System Aplicado:** Respeito estrito à paleta de cores (Primary Orange, Secondary Yellow, Dark Background) e à tipografia **Inter**.

- Tela 1:** Feed de Eventos (Foco na clareza para o comprador).

- Tela 2:** Listar eventos do organizador (Foco gerencial).



Considerações finais



- A refatoração transformou um código rígido numa arquitetura profissional, modular e escalável.
- A separação de responsabilidades (Model, View, Controllers) facilitou drasticamente a manutenção e a implementação de testes.
- O protótipo visual tangibilizou o valor do produto para o usuário final.

Referências



- Documentação Oficial do Kotlin (kotlinlang.org)
- Figma - Ferramenta de Prototipação Colaborativa
- Material de Apoio da Disciplina: "Outras Atividades 1 - OAT2 - 2026.1"